# ALGORITM DE STRUCTURARE TEMPLATE MICROINSTRUCȚIUNE

$\mu SB = (\mu_B(\mu_O), \prec)$

↓

Algoritm partiționare: $\mu PT = \{\mu i_1, \dots, \mu i_{|\mu PT|}\}$

↓

Algoritm de formare template $\mu i$

## Algoritm de formare template $\mu i$

1. Se formează tabela de incompatibilitate
2. Generăm toate MIC și toate MCC din tabela obținută la 1
3. Alegem clasa cu cardinal maxim din MIC

   $MIC_m \in \{MIC\}$ a.î. $|MIC|_m > |MIC_j| \ \forall j \in 1, |MIC| \ j \neq m$
4. Generăm AMCC asociate lui $MIC_m$

   $AMCC_m = \{MCC \mid \forall \mu_O \in MIC_m, \mu_O \in MCC\}$
5. Generăm tabela de acoperire modificată $\Rightarrow$ TAM

   $TAM: AMCC_m \times \mu_B(\mu_O) \to B$ (B: 0/1)
6. Alegem mulțimea de MCC care sunt esențiale

   $\{MCC\}_e \in AMCC_m$

   $\exists \mu_O \in \{MCC\}_e$ a.î. $TAM(MCC_e, \mu_O) = 1$
7. Generăm tabela de acoperire redusă $\to$ TAR

   $TAR: AMCC_m \times (\mu_B(\mu_O) \setminus MIC_m \setminus \{MCC\}_e) \to B$
8. Se generează SOL

   $SOL = \{SOL_1, \dots, SOL_K\}$

   $SOL_j = \wedge / \{MCC\}_e \cup \{MCC\}_{acoperire}$
9. Se generează soluțiile complete

   $SOLC_j = SOL_j \wedge \{MCC\}_{neacoperite}$